ISTQB CTFL v4.0 — Complete Exam Prep Guide

ISTQB® Certified Tester Foundation Level (CTFL v4.0) — Complete Exam Prep Guide

Exam format: 40 multiple-choice questions · 60 minutes (75 for non-native English speakers) · Pass mark: 26/40 (65%) · 1 point per question · Closed book

Syllabus chapters and question distribution (approx.):

Chapter	Topic	Questions
1	Fundamentals of Testing	8
2	Testing Throughout the SDLC	6
3	Static Testing	4
4	Test Analysis and Design	11
5	Managing the Test Activities	9
6	Test Tools	2

K-Levels (cognitive levels): K1 = Remember (recall a term), K2 = Understand (explain, compare, classify), K3 = Apply (use a technique on a scenario — e.g., derive test cases with boundary value analysis).

CHAPTER 1 — FUNDAMENTALS OF TESTING

1.1 What is Testing?

Software testing is a **set of activities to discover defects and evaluate the quality** of software artifacts (called **test objects**). Testing is NOT just executing software and checking results — it includes planning, analysis, design, implementation, reporting, and evaluation.

Key clarifications:

- **Dynamic testing** = testing that involves executing the software.
- **Static testing** = testing that does NOT execute the software (reviews, static analysis).
- Testing is not only about **verification** (checking against requirements/specifications — "did we build the product right?") but also **validation** (checking the system meets users' real needs — "did we build the right product?").
- **Testing ≠ Debugging.** Testing *finds* failures/defects. Debugging is a *development* activity: reproducing the failure, diagnosing the root cause (finding the defect), and fixing it. **Confirmation testing** (retesting) afterward confirms the fix worked — and that's testing again.

1.1.1 Test Objectives (typical)

- Evaluate work products (requirements, user stories, design, code)
- Cause failures and find defects
- Ensure required coverage of a test object
- Reduce the level of risk of inadequate software quality
- Verify whether specified requirements have been fulfilled
- Verify compliance with contractual, legal, regulatory requirements
- Provide information to stakeholders for informed decisions
- Build confidence in the quality of the test object
- Validate whether the test object works as users and stakeholders expect

1.1.2 Testing vs. Debugging

Testing	Debugging
Triggers failures (dynamic) or finds defects directly (static)	Finds root cause, analyzes it, fixes it
A test activity	A development activity
After a fix: **confirmation testing (retesting)** verifies the fix; **regression testing** checks nothing else broke	—

1.2 Why is Testing Necessary?

Testing (as part of QA) helps achieve agreed goals within scope, time, quality, and budget constraints. **Everyone can and should test** (developers, analysts, product owners) — but dedicated testers bring focused skills and independence.

1.2.1 Testing's Contributions to Success

- Cost-effective way to detect defects early (cheap to fix early)
- Directly evaluates quality; supports go/no-go release decisions
- Represents the users' perspective within the project
- Legal/contractual compliance evidence

1.2.2 Testing and Quality Assurance (QA)

- **Testing** is a form of **quality control (QC)** — a *product-oriented, corrective* approach: finding defects in results.
- **QA** is *process-oriented, preventive*: improving processes so good processes yield good products. QA applies to both development and testing processes.
- Test results are used by QC (to fix defects) and by QA (as feedback on process performance).

1.2.3 Errors, Defects, Failures, and Root Causes — MEMORIZE THIS CHAIN

- **Error (mistake):** A human action producing an incorrect result (e.g., programmer misunderstands a requirement).

- **Defect (fault, bug):** An imperfection in a work product (code, document) resulting from an error.
- **Failure:** An event where the system deviates from expected behavior — occurs when a defect in code is *executed*.
- Not every defect causes a failure (some code never executes). Failures can also be caused by environmental conditions (radiation, magnetism, hardware) — not just defects.
- **False positive:** A test reports a defect that isn't there (e.g., test itself is wrong). **False negative:** A test fails to detect a real defect.
- **Root cause:** The fundamental reason for the error (e.g., ambiguous requirement, time pressure, lack of training). Root cause analysis prevents *future* similar defects.

Chain: **Root cause → Error → Defect → (execution) → Failure**

1.3 Testing Principles (THE 7 PRINCIPLES — heavily tested!)

1. **Testing shows the presence of defects, not their absence.** Testing can prove defects exist, but can never prove there are none. Testing reduces the probability of undiscovered defects but is not a proof of correctness.
2. **Exhaustive testing is impossible.** Testing everything (all inputs, all preconditions) is infeasible except in trivial cases. Use risk analysis, test techniques, and priorities to focus effort.
3. **Early testing saves time and money.** (Shift-left.) Start static and dynamic testing as early as possible — defects found early are cheaper to fix.
4. **Defects cluster together.** A small number of modules usually contains most of the defects (Pareto principle ~80/20). Predicted/observed clusters are an input to risk-based testing.
5. **Tests wear out (pesticide paradox).** Repeating the same tests stops finding new defects. Tests, test data, and techniques need to be revised/updated. (Exception: repeated regression tests can still show a benefit by confirming stability.)
6. **Testing is context dependent.** There is no single universal approach — a safety-critical control system is tested differently from an e-commerce app.
7. **Absence-of-defects fallacy.** Verifying all specified requirements and fixing all found defects does NOT guarantee success — the system may still fail to meet users' needs and expectations (validation matters!).

1.4 Test Activities, Testware and Test Roles

1.4.1 Test Activities and Tasks (the test process)

The main activity groups (not necessarily sequential — often iterative/overlapping):

1. **Test planning** — define objectives, select approach, schedule, decide exit criteria.
2. **Test monitoring and control** — ongoing comparison of actual vs. planned progress; corrective actions.
3. **Test analysis** — "**what to test?**" Analyze the test basis to identify testable features, define and prioritize **test conditions**, evaluate risks.
4. **Test design** — "**how to test?**" Elaborate test conditions into **test cases** (high-level), define test data needs, test environment needs, identify coverage items.
5. **Test implementation** — create/acquire testware needed for execution: concrete test cases with data, **test procedures**, test scripts, test suites; build the environment; organize execution schedule.
6. **Test execution** — run tests (manually or automated), compare actual vs. expected results, log results, report **anomalies/defects**.
7. **Test completion** — at project milestones/release: archive testware, hand over to maintenance teams, analyze lessons learned, write test completion report.

1.4.2 Test Process in Context

Testing is not standalone — context factors influence the test process: stakeholders, team skills, business domain, technical factors (technology, architecture), project constraints (scope, time, budget, resources), organizational factors, SDLC, tool availability.

1.4.3 Testware (work products created by test activities)

- From planning: test plan, schedule, risk register, entry & exit criteria
- From monitoring & control: test progress reports, control directives, risk info
- From analysis: (prioritized) test conditions (e.g., test charters in exploratory testing), defect reports for test basis defects
- From design: (prioritized) test cases, test charters, coverage items, test data requirements, test environment requirements

- From implementation: test procedures, automated test scripts, test suites, test data, test execution schedule, test environment elements (stubs, drivers, simulators, service virtualizations)
- From execution: test logs, defect reports
- From completion: test completion report, action items for improvement, archived testware

1.4.4 Traceability

Maintain traceability between test basis elements ↔ test conditions ↔ test cases ↔ results/defects. Benefits: coverage evaluation, impact analysis of changes, test auditability, meeting IT governance criteria, making reports understandable (e.g., “requirement X passed”).

1.4.5 Roles in Testing

- **Test management role:** overall responsibility for the test process, test team, leadership of test activities — planning, monitoring, control, completion.
- **Testing role:** responsibility for the engineering (technical) side — analysis, design, implementation, execution. Roles ≠ people: one person can hold both roles; roles can be shared across a team.

1.5 Essential Skills and Good Practices in Testing

1.5.1 Generic Skills for Testers

Testing knowledge, thoroughness/carefulness/curiosity/attention to detail, good communication + active listening + being a team player, analytical & critical thinking, creativity, technical knowledge, domain knowledge.

Testers deliver “bad news” (defects) — need diplomacy: communicate defects constructively, focus on facts and benefits, not blame.

1.5.2 Whole Team Approach

(From Extreme Programming.) Any team member with the necessary skills can perform any task; everyone is responsible for quality. Testers work closely with developers and business reps; co-location; shared responsibility. May not fit contexts requiring high independence (e.g., safety-critical).

1.5.3 Independence of Testing

Levels of independence (low → high):

1. No independence: author tests own work
2. Author’s peers (same team)
3. Testers outside the team but within the organization (independent test team)
4. Testers external to the organization

Benefits: independent testers see different defects; unbiased; can challenge assumptions. **Drawbacks:** isolation from dev team, communication problems, adversarial “us vs. them,” developers may lose sense of responsibility for quality, bottlenecks.

CHAPTER 2 — TESTING THROUGHOUT THE SOFTWARE DEVELOPMENT LIFECYCLE

2.1 Testing in the Context of a Software Development Lifecycle (SDLC)

2.1.1 Impact of the SDLC on Testing

- **Sequential models (waterfall, V-model):** In initial phases testers usually participate in reviews (requirements, design). Dynamic testing typically starts late, near the end.
- **Iterative (spiral, prototyping) and incremental (e.g., Unified Process) models:** Software grows in increments; each delivery needs both new-feature testing and **regression testing** of everything already delivered.
- **Agile:** Short iterations; testing continuous throughout; heavy automation of regression; testers embedded in the team; change is welcomed.

The SDLC affects: scope & timing of test activities, level of detail of documentation, choice of techniques and approach, extent of automation, tester's role & responsibilities.

2.1.2 SDLC and Good Testing Practices

- **Every development activity has a corresponding test activity** (so every level is tested with quality control).
- **Different test levels have specific, different objectives** (avoids redundancy, no gaps).
- **Test analysis & design for a level begins during the corresponding development phase** (early testing / shift-left).
- **Testers participate in discussions/reviews of work products as soon as drafts exist** (defect prevention).

2.1.3 Testing as a Driver for Software Development — TDD, ATDD, BDD

All three are **test-first** approaches (tests before implementation), closely tied to early testing and shift-left. Tests act as executable specifications.

- **TDD (Test-Driven Development):** Developer writes small automated tests (usually unit level) → writes code to pass them → refactors. Directs coding through test cases.
- **ATDD (Acceptance Test-Driven Development):** Tests derived from acceptance criteria during system design; written before the corresponding part is developed; often become acceptance tests.
- **BDD (Behavior-Driven Development):** Expresses desired behavior in natural-language form understandable to all stakeholders — typically **Given / When / Then** (Gherkin format). Test cases then translated into automated tests.

2.1.4 DevOps and Testing

DevOps = culture + practices bringing Dev and Ops together, enabled by **CI/CD** (continuous integration / continuous delivery).

Benefits for testing: fast feedback on code quality; CI promotes shift-left (developers submit code with automated component/integration tests); automated processes like CI/CD enable stable test environments; more focus on non-functional characteristics; automation via delivery pipeline reduces repetitive manual testing; regression risk minimized by scale of automated regression tests.

Risks/challenges: the delivery pipeline must be defined and established; CI/CD tools must be introduced and maintained; automation requires additional resources, can be hard to build and maintain.

DevOps does NOT eliminate manual testing — exploratory testing, usability, user-facing acceptance still needed.

2.1.5 Shift-Left

Performing testing earlier in the SDLC. Examples:

- Reviewing the specification from the testers' perspective (finds ambiguities early)
- Writing test cases before code, running them in a test harness during implementation
- Using CI/CD with fast feedback and automated component tests
- Static analysis of source code before dynamic testing (or as part of an automated pipeline)
- Performing non-functional testing at component level where possible

Shift-left may mean extra training/effort/cost earlier, but saves effort/cost later.

2.1.6 Retrospectives and Process Improvement

Held at the end of a project/iteration ("post-project meetings"): discuss what was successful, what could be improved, how to embed improvements. Benefits for testing: increased test effectiveness/efficiency, higher testware quality, team bonding & learning, improved test basis quality, better dev–test cooperation.

2.2 Test Levels and Test Types

2.2.1 Test Levels — MEMORIZE

Test levels are groups of test activities organized and managed together, tied to development stages. Each level has: specific objectives, test basis, test object, typical defects/failures, distinct approaches & responsibilities.

1. **Component (unit) testing** — tests components in isolation. Usually done by developers, needs test harnesses/frameworks (e.g., xUnit). Focus: functional (e.g., correct calculations) and non-functional (e.g., memory leaks) plus structural properties.
2. **Component integration testing** — tests interfaces & interactions *between components*. Heavily dependent on integration strategy (bottom-up, top-down, big-bang).
3. **System testing** — the whole system's behavior and capabilities, end-to-end tasks, functional + non-functional (often in a test environment mirroring production). Often by an independent test team. May rely on specifications.
4. **System integration testing** — interfaces of the system with other systems and external services (e.g., third-party web services). Needs suitable environments, ideally production-like.
5. **Acceptance testing** — validation: is the system fit for use, ready for deployment? Mainly the users'/customers' responsibility. Forms include:
   - **User acceptance testing (UAT)** — fit for users' business needs
   - **Operational acceptance testing** — backup/restore, install/upgrade, disaster recovery, security, maintenance (ops/admin staff)
   - **Contractual & regulatory acceptance** — against contract criteria / regulations (e.g., government, legal, safety)
   - **Alpha testing** — at the developing organization's site, by potential users
   - **Beta testing** — at users' own locations

2.2.2 Test Types — MEMORIZE THE 4

1. **Functional testing** — "**what** the system does." Evaluates functions against functional requirements. Completeness, correctness, appropriateness.
2. **Non-functional testing** — "**how well** the system behaves." Evaluates quality characteristics per ISO/IEC 25010: performance efficiency, compatibility, usability (interaction capability), reliability, security, maintainability, portability (flexibility), safety. Non-functional testing can and should start early (shift-left). Often needs special techniques (e.g., boundary values for performance limits) and tools.
3. **Black-box testing** — specification-based; derives tests from documented external behavior; measures coverage of the specification, not code.
4. **White-box testing** — structure-based; derives tests from the internal structure (code, architecture, data flow); coverage measured against structural elements (statements, branches).

All four test types can be applied at every test level. Example: functional test at component level; security (non-functional) at component level; branch coverage (white-box) at system level (of menus).

2.2.3 Confirmation Testing and Regression Testing

- **Confirmation testing (retesting):** After a defect fix — re-run the failed test (and possibly new tests for the fix) to confirm the defect is fixed. In resource crunch, may be reduced to just reproducing the failing steps.
- **Regression testing:** Confirm that a change (fix or new feature) has NOT caused adverse consequences elsewhere ("regressions"). Advised to do impact analysis to optimize the extent. Regression suites grow → run many times → strong candidates for **automation**, starting early. In CI/CD (DevOps), automated regression tests ideally run at every commit/stage.
- Both are done at all test levels whenever defects are fixed or code changes.

2.3 Maintenance Testing

Maintenance = changes to a delivered/operational system. Categories (ISO/IEC 14764): **corrective** (fixes), **adaptive** (adapt to environment changes — new OS, new regulations), **perfective** (improvements, performance).

Maintenance testing scope depends on: **degree of risk of the change**, **size of the existing system**, **size of the**

change.

Triggers for maintenance testing:

- **Modifications** — planned enhancements, corrective/emergency fixes, patches
- **Environment upgrades/migrations** — new platform (test the changed software + operational tests in the new environment); data migration/conversion testing when data moves from another application
- **Retirement** — of a system; may need testing of data migration/archiving for long data-retention periods

Maintenance testing typically = testing the changes + regression testing of unchanged parts (impact analysis helps scope it).

CHAPTER 3 — STATIC TESTING

3.1 Static Testing Basics

Static testing examines work products WITHOUT executing them. Two forms:

- **Reviews** (manual examination — informal to formal)
- **Static analysis** (tool-driven examination of code/models; essential in safety-critical systems; in DevOps often part of the pipeline; includes linters, security scanners, complexity checkers)

3.1.1 Work Products Examinable by Static Testing

Almost any readable work product: requirements documents, user stories & acceptance criteria, code, test plans/testware, contracts, models, configuration files, website content, schedules/budgets. Anything that can be *read and understood*. (Not suitable: work products that can't be meaningfully read by humans, e.g., executable-only third-party code — though static analysis tools may still process code.)

3.1.2 Value of Static Testing

- Detects defects **early** — cheapest point to fix (early testing principle)
- Finds defects that dynamic testing **cannot** find (e.g., unreachable code, deviations from coding standards, defects in requirements — ambiguities, contradictions, omissions, inconsistencies, redundancies)
- Builds shared understanding; improves communication among stakeholders
- Reviews cheaply detect missing requirements (dynamic testing can't easily reveal what's absent)

3.1.3 Static vs. Dynamic Testing — differences

Static	Dynamic
No execution	Requires execution
Finds **defects directly** in work products	Finds **failures**, then defects deduced via debugging
Can assess non-executable artifacts (requirements, contracts)	Only executable test objects
Finds: inconsistencies, ambiguities, contradictions, omissions, inaccuracies, redundancy; unreachable code; standard violations; security vulnerabilities (some)	Finds: behavior deviations, performance issues, anything observable only at runtime
Measures quality attributes not execution-dependent (maintainability)	Measures runtime quality (performance, reliability)

Both approaches are complementary — typical defects found by each differ.

3.2 Feedback and Review Process

3.2.1 Benefits of Early and Frequent Stakeholder Feedback

Prevents misunderstanding requirements; catches requirement changes early; ensures dev team understands what to build; reduces risk of building the wrong product; Agile methods institutionalize this (frequent iterations, demos, reviews of each increment).

3.2.2 Review Process Activities (per ISO/IEC 20246) — MEMORIZE ORDER

1. **Planning** — define scope (purpose, work product, quality characteristics to evaluate), effort, timeframes, review characteristics (type, roles, activities, checklists)
2. **Review initiation** — ensure everyone/everything is prepared: distribute work product & materials, explain scope/objectives/roles, answer questions
3. **Individual review** — each reviewer examines the work product alone; identifies **anomalies**, recommendations, questions; logs them (e.g., in a tool or on the document)
4. **Communication and analysis** — a review meeting (or other communication) discusses anomalies: is each a defect? Decide status, ownership, actions; quality-level decision for the work product (accept / accept with changes / reject-rework + new review)
5. **Fixing and reporting** — defect reports created; defects fixed (by author); communicate defects to the right person/team; record review results/metrics

3.2.3 Roles in Reviews — MEMORIZE

- **Manager** — decides what is reviewed; provides resources (staff, time)
- **Author** — creates and fixes the work product under review
- **Moderator (facilitator)** — ensures effective running of meetings, mediation, time management, a safe environment where everyone can speak freely
- **Scribe (recorder)** — collates anomalies from reviewers; records meeting info: decisions, new anomalies found in the meeting
- **Reviewer** — performs the review; can be someone on the project, a subject-matter expert, or a stakeholder
- **Review leader** — overall responsibility for the review: who participates, when/where the review takes place

(One person may take several roles; roles may merge depending on review type. Tools may support the scribe role.)

3.2.4 Review Types — MEMORIZE (informal → formal)

1. **Informal review** — no defined process, no meeting required, may be done by a colleague (“buddy check”) or several people; main aim: **detect anomalies** cheaply/quickly.
2. **Walkthrough** — **led by the AUTHOR**; individual preparation of reviewers optional; goals: find defects, improve the product, consider alternatives, evaluate conformance, exchange knowledge, train participants; scribe is mandatory.
3. **Technical review** — performed by **technically qualified reviewers**; **moderator** leads (ideally trained, NOT the author); goals: gain consensus, make decisions on technical problems, detect anomalies, evaluate quality, build confidence, generate new ideas.
4. **Inspection** — **the MOST FORMAL** review type: follows the complete generic process; rule/checklist-based with formal documented outputs; strict roles (may add a *reader*); main objective: **find the maximum number of anomalies**; also process improvement via metrics + root-cause analysis; **author cannot act as moderator, reader, or scribe**; metrics collected & used to improve the SDLC and review process.

3.2.5 Success Factors for Reviews

- Clear **objectives** & measurable exit criteria defined up front (evaluation of participants must NEVER be an objective!)
- Choose the review type appropriate to objectives, work product type, participants, project needs
- Review small chunks — don’t overload reviewers’ attention (limited individual review sessions; take breaks)
- Provide feedback to stakeholders/authors so they improve the work products & process
- Adequate **time** for preparation
- **Management support**
- Reviews integrated in the organization’s culture — promoting learning and process improvement
- Training in review techniques (especially formal ones)
- Meetings well-managed — participants see it as valuable use of time
- Right people involved; testers as valued reviewers (they learn the product, and prepare tests earlier)
- Psychological safety: defects are welcomed, expressed objectively; no blame on the author

---

CHAPTER 4 — TEST ANALYSIS AND DESIGN

4.1 Test Techniques Overview

- **Black-box techniques** (specification-based): based on required behavior; tests don't depend on implementation, so they remain valid if implementation changes but behavior doesn't.
- **White-box techniques** (structure-based): based on internal structure (code, architecture); coverage measured against structure.
- **Experience-based techniques:** use testers' knowledge & experience (of the software, its environment, likely defects). Effective complement to systematic techniques; skill-dependent.

4.2 Black-Box Test Techniques

4.2.1 Equivalence Partitioning (EP) — K3 (be able to apply!)

- Divide data into **partitions (equivalence classes)** where all members are expected to be processed the same way.
- Partitions exist for **valid** values (accepted) and **invalid** values (rejected).
- Partitions must be: derived from the value set; **non-empty, disjoint (non-overlapping), and complete** (cover the whole set).
- **Any single value from a partition represents the whole partition.**
- **Each Choice coverage:** every partition must be exercised at least once. Coverage = (partitions exercised ÷ total partitions) × 100%.
- When testing multiple parameter partitions together: valid partitions of different parameters may be combined in one test, but an invalid partition should usually be tested **individually** (one invalid at a time) so failures aren't masked.

Example: Field accepts integers 1–100. Partitions: P1 = <1 (invalid), P2 = 1–100 (valid), P3 = >100 (invalid). Minimum tests for 100% Each-Choice coverage: 3 (e.g., 0, 50, 101).

4.2.2 Boundary Value Analysis (BVA) — K3

- Applies only to **ordered partitions**; tests the **boundaries** (minimum & maximum of a partition), because developers commonly err there (e.g., `<` vs `<=`).
- **2-value BVA:** for each boundary, test the boundary value + its closest neighbor in the adjacent partition. Coverage item = each such value.
- **3-value BVA:** for each boundary, test the boundary and BOTH neighbors (before and after). More rigorous — catches defects like `if (x = 10)` mistyped for `if (x ≥ 10)` that 2-value BVA might miss.

Example: valid range 1–100. 2-value BVA values: 0, 1, 100, 101. 3-value BVA values: 0, 1, 2, 99, 100, 101.

4.2.3 Decision Table Testing — K3

- For system behavior driven by **combinations of conditions** (business rules).
- Columns = rules; rows = **conditions** (top) and resulting **actions** (bottom).
- Notation: T/F (or 1/0) for conditions; X for action performed; "–" (dash) = condition value irrelevant ("don't care"); N/A = infeasible.
- **Full decision table:** all combinations = 2^n columns for n boolean conditions. Then **collapse/minimize**: merge columns where some conditions don't affect the outcome.
- Detects gaps/contradictions in requirements. **Coverage: each column (rule) with a feasible combination of conditions must be exercised ≥ once.** Coverage = exercised columns ÷ feasible columns.

4.2.4 State Transition Testing — K3

- Model: **state transition diagram/table** — states, transitions, events, guard conditions, actions. Transition caused by an event; may have guard condition; may trigger an action.
- **State table:** rows = states, columns = events (+guards); cells = resulting state (+actions); empty cell = **invalid transition**.
- Coverage criteria:
  - **All-states coverage:** every state visited ≥ once. (Weakest — coverage = visited states ÷ total states)
  - **Valid transitions coverage (0-switch):** every valid transition exercised ≥ once. **Most widely used.** Full valid-transitions coverage guarantees full all-states coverage.
  - **All-transitions coverage:** ALL transitions in the state TABLE — valid AND invalid (test that invalid events are

properly rejected). Full all-transitions coverage guarantees both of the above. Usually feasible only for small models.

4.2.5 (Related) Use Case Testing — tests derived from use cases: exercise basic (main) flow, alternative flows, exception flows. (Mentioned across syllabus versions; know the concept.)

4.3 White-Box Test Techniques

4.3.1 Statement Testing / Statement Coverage

- Coverage items = executable **statements**.
- Coverage = (statements executed ÷ total executable statements) × 100%.
- 100% statement coverage ensures every statement ran at least once — but NOT every decision outcome (e.g., an `if` without `else`: one test through the "true" path gives 100% statement coverage but never tests the false outcome).

4.3.2 Branch Testing / Branch Coverage

- A **branch** = a transfer of control between two nodes in the control flow graph (conditional branches from decisions: if, switch, loops; and unconditional ones).
- Coverage = (branches exercised ÷ total branches) × 100%.
- **Branch coverage SUBSUMES statement coverage:** 100% branch coverage ⇒ 100% statement coverage (never vice versa).

4.3.3 Value of White-Box Testing

- Fundamental strength: the **entire implementation** can be considered — tests can find defects in code that specification-based tests would miss (code with no corresponding specification).
- Weakness: if the specification is not implemented at all (**missing functionality / defects of omission**), white-box testing won't find it — it only tests code that exists.
- Coverage measurement gives objective evidence, useful even during black-box testing (measure code coverage achieved, add white-box tests for uncovered areas).
- Widely used in component testing/TDD; mandated coverage levels in safety standards (e.g., avionics requires MC/DC at highest integrity levels).

4.4 Experience-Based Test Techniques

4.4.1 Error Guessing — K2

- Anticipate errors/defects/failures based on tester's knowledge: how the app worked in the past, typical developer errors, similar applications, failures elsewhere.
- **Fault attacks:** a structured form — make/use lists of possible errors, defects & failures and design tests to expose them.
- Typical error sources: input (invalid, missing), output, logic, computation, interfaces, data.

4.4.2 Exploratory Testing — K2

- Tests are simultaneously **designed, executed, and evaluated (learned from)** while the tester learns the test object — unscripted, concurrent learning/design/execution/evaluation.
- **Session-based** approach makes it manageable: time-boxed sessions (e.g., 60–120 min), guided by a **test charter** (objectives), documented in **session sheets**; debriefing afterward.
- Best when: specifications are poor/missing, time pressure, complement to formal techniques. Effectiveness depends on tester's experience, domain knowledge, and skills (analytical, curiosity, creativity).

4.4.3 Checklist-Based Testing — K2

- Design/execute tests to cover items on a **checklist** (of test conditions), built from experience, user knowledge, known failure reasons.
- Checklists shouldn't be too long or too detailed; items often phrased as questions; should be checkable directly.
- Checklists need **maintenance** (updated with new defects, pruned of obsolete items). Provide consistency + some flexibility; risk: without detailed steps, results may vary between testers.

4.5 Collaboration-Based Test Approaches

(Defect PREVENTION through collaboration — vs. techniques above which focus on defect detection.)

4.5.1 Collaborative User Story Writing

- User story = smallest item of value; typical format: **“As a [role], I want [goal], so that [benefit].”**
- The **3 C’s: Card** (the story’s written form), **Conversation** (how the software will be used — the discussions matter more than the card), **Confirmation** (the acceptance criteria).
- **INVEST** criteria for good stories: **I**ndependent, **N**egotiable, **V**aluable, **E**stimable, **S**mall, **T**estable.
- Collaborative authorship (e.g., three amigos: business rep + developer + tester) detects defects/ambiguities before coding.

4.5.2 Acceptance Criteria

- Conditions a user story must meet to be accepted — define scope, are the test basis, enable accurate estimation.
- Common formats: **scenario-oriented** (Given/When/Then, BDD style) and **rule-oriented** (bullet list of rules / input-output table). Any custom format is fine if clear and unambiguous.

4.5.3 Acceptance Test-Driven Development (ATDD) — K3 (be able to derive tests from acceptance criteria!)

- Test-first: acceptance tests created **before** the story is implemented, usually in a specification workshop while analyzing/refining the story.
- Tests written by team members with different perspectives (customer, dev, tester).
- Positive tests first (expected behavior, no exceptions), then negative tests, then non-functional aspects.
- Tests must be understandable by stakeholders — plain language sentences (natural language or a BDD framework); they become executable specifications and can drive development.

CHAPTER 5 — MANAGING THE TEST ACTIVITIES

5.1 Test Planning

5.1.1 Purpose and Content of a Test Plan

The test plan documents test objectives, resources, and processes; declares exit criteria; helps thinking through challenges; is a means of communicating with the team & stakeholders; demonstrates testing adheres to the test policy/strategy.

Typical content: context of testing (scope, objectives, constraints, test basis); assumptions & constraints; stakeholders (roles, responsibilities, relevance to testing, hiring/training needs); communication (forms/frequency of communication, documentation templates); risk register (product risks, project risks); test approach (test levels, types, techniques, deliverables, entry & exit criteria, independence level, metrics, test data & environment requirements, deviations from org test policy/strategy); budget & schedule.

5.1.2 Tester's Contribution to Iteration and Release Planning (Agile)

- **Release planning** (longer horizon, defines/redefines product backlog): testers participate in writing testable user stories & acceptance criteria, do risk analysis, estimate test effort for stories, determine the test approach, plan release-level testing.
- **Iteration planning** (single iteration): testers do detailed risk analysis of stories, estimate testability, break stories into tasks (esp. testing tasks), estimate test effort for tasks, identify functional & non-functional aspects to test.

5.1.3 Entry and Exit Criteria

- **Entry criteria** (Agile: "Definition of Ready") — preconditions to start an activity: resources availability (people, tools, environments, data, budget, time), testware availability (test basis, testable requirements, user stories, test cases), initial quality level (e.g., all smoke tests passed).
- **Exit criteria** (Agile: "Definition of Done") — what must be achieved to declare completion: measures of thoroughness (coverage achieved, number of unresolved defects, defect density, failure rate) and completion criteria (planned tests executed, static testing performed, all found defects reported, all regression tests automated).
- Note: running out of **time or budget** can also legitimately end testing — provided stakeholders accept the residual risk.

5.1.4 Estimation Techniques — MEMORIZE THE 4

1. **Estimation based on ratios (metrics-based):** use historical ratios from the organization (e.g., dev:test ratio 3:2 → dev effort 600 person-days ⇒ test ≈ 400).
2. **Extrapolation (metrics-based):** measure early in the current project, extrapolate the remaining effort (e.g., average effort of last 3 iterations → next iteration). Fits Agile well.
3. **Wideband Delphi (expert-based):** experts estimate independently → discuss deviations → re-estimate until consensus. **Planning Poker** (with cards, often Fibonacci values) is the Agile variant.
4. **Three-point estimation (expert-based):** most optimistic (a), most likely (m), most pessimistic (b). **E = (a + 4m + b) / 6**, standard deviation **SD = (b − a) / 6**. Example: a=6, m=9, b=18 → E = (6+36+18)/6 = 10; SD = (18−6)/6 = 2 → estimate 10 ± 2 person-hours.

5.1.5 Test Case Prioritization Strategies

- **Risk-based prioritization:** run tests covering the most important risks first.
- **Coverage-based prioritization:** run tests achieving higher coverage first (e.g., additional-coverage variant: order by *additional* coverage gained).
- **Requirements-based prioritization:** run tests for the most important requirements first (priority set by stakeholders).
- Constraint: **dependencies** (test A needed before B; resource availability) may override the ideal priority order.

5.1.6 Test Pyramid

- Layers of automated tests: **bottom = component/unit tests** (many, small, isolated, FAST, cheap), middle = integration/service/API tests, **top = end-to-end/UI tests** (few, large scope, SLOW, expensive/brittle).
- Lower layers → high granularity, fast execution → more tests there; top layers → low granularity/high complexity → fewer tests.

5.1.7 Testing Quadrants (Brian Marick / Agile Testing)

Group tests by: **business-facing vs technology-facing** × **support the team (guide development) vs critique the product**.

- **Q1** — technology-facing, support team: component & component-integration tests, automated, CI.
- **Q2** — business-facing, support team: functional tests, examples, user story tests, API tests, prototypes — often automated (ATDD).
- **Q3** — business-facing, critique product: exploratory testing, usability, UAT — user-oriented, often manual.
- **Q4** — technology-facing, critique product: non-functional tests (performance, security, reliability… but NOT usability which is Q3) — often tool-supported.

5.2 Risk Management

5.2.1 Risk Definition and Attributes

- **Risk** = a potential event/hazard/threat whose occurrence causes an adverse effect. Two factors: **risk likelihood** (probability, 0<p<1) and **risk impact (harm)**. **Risk level = likelihood × impact.**

5.2.2 Project Risks vs. Product Risks — MEMORIZE THE DISTINCTION

- **Project risks** — affect the project's ability to achieve its objectives (delays, budget overruns): organizational issues (delays, inaccurate estimates, cost-cutting), people issues (skills gaps, conflicts, communication, staff shortage), technical issues (scope creep, poor tool support), supplier issues (delivery failure, company insolvency).
- **Product risks** — related to product quality; potential failure of the product to satisfy user/stakeholder needs: missing/wrong functionality, incorrect computations, runtime errors, poor architecture, inefficient algorithms, poor performance/security/usability. Materialized product risks cause: user dissatisfaction, loss of revenue/trust/reputation, damages, legal penalties, criminal liability in extreme cases, harm — even injury or death (safety-critical).

5.2.3 Product Risk Analysis

= risk **identification** + risk **assessment** (categorize, determine likelihood & impact & level, prioritize, propose handling). Goal: focus testing so residual risk is minimized. Begins **early**. Influences: thoroughness & scope of testing; which test levels/types to perform; which techniques & coverage to use; prioritization (find critical defects early); whether non-testing actions can reduce risk (e.g., training).

5.2.4 Product Risk Control

All measures taken against identified & assessed product risks: **risk mitigation** (actions from risk assessment to reduce level, e.g., testing) and **risk monitoring** (check mitigation works, gather new info to refine assessments, identify new/emerging risks). Testing responses: select experienced testers, apply independence, perform reviews & static analysis, apply the appropriate techniques & coverage, apply the right test types (quality characteristics), perform dynamic testing including regression testing.

5.3 Test Monitoring, Test Control and Test Completion

- **Test monitoring:** gathering information about testing — assess progress, check exit criteria/DoD (coverage of risks/requirements/acceptance criteria achieved?).
- **Test control:** using that information to issue **corrective/guiding directives**: reprioritize tests, re-evaluate entry/exit criteria, adjust schedule, add resources.
- **Test completion:** collects data from completed test activities (at milestones: release, end of iteration, end of a level) to consolidate experience, testware, and other relevant information.

5.3.1 Common Metrics

Project progress (task completion, resource usage, test effort); test progress (implemented/executed/passed/failed test cases; execution time); product quality (availability, response time, MTTF); defect metrics (number & priority of defects found/fixed, defect density, defect detection percentage); risk coverage (incl. residual risk level); coverage metrics (requirements coverage, code coverage); cost metrics (cost of testing, cost of quality).

5.3.2 Purpose, Content, Audience of Test Reports

- **Test progress report** — regular, DURING testing, supports ongoing control; audience mostly the team & test manager. Typical content: period covered, progress vs plan (deviations), impediments, metrics, new/changed risks, testing planned for next period.

- **Test completion report** — at the END (of project/level/iteration); summarizes a stage; gives info for later testing; audience: stakeholders. Typical content: test summary, evaluation of testing & product quality vs objectives, deviations from plan, testing metrics, unmitigated/residual risks & unfixed defects, lessons learned & reusable testware.
- **Tailor to the audience:** business stakeholders → residual risks, quality statements, plain language; technical audiences → defect detail, technical metrics. Ad hoc reporting may be needed anytime (in Agile, may be built into task boards & daily standups).

5.3.3 Communicating the Status of Testing

Ways: verbal (daily stand-ups, meetings), dashboards (CI/CD dashboards, task boards, burn-down charts), electronic channels, documentation, formal reports. Choose per stakeholders' needs & organizational context.

5.4 Configuration Management (CM)

- Purpose in testing: identify, control, track versions of **configuration items** — the test object and all testware (test plans, cases, scripts, results, environments, tools).
- All items uniquely identified, version controlled, changes tracked, related to each other (+ to versions of the test object) so **traceability & reproducibility** are maintained throughout.
- Supports determining exactly what was tested against which version.
- In DevOps, CM is usually part of the CI/CD pipeline (automated: version control, build automation, deployment configuration).

5.5 Defect Management

- Purpose: handle **anomalies** (may or may not be actual defects) from discovery through classification, investigation, resolution, and closure.
- Process needs: a workflow + rules for classification. Anomalies may be rejected as false positives, or become change requests, or be confirmed defects.
- A typical **defect report** for dynamic testing contains:
  - Unique identifier; title/short summary
  - Date, author, author's role, test item & environment, SDLC phase where observed
  - Description enabling **reproduction** (steps, logs, screenshots, dumps)
  - Expected result vs. actual result
  - Severity (impact on the system/stakeholders) & priority (urgency of fixing — business importance)
  - Status (e.g., open, deferred, duplicate, waiting to be fixed, fixed awaiting confirmation testing, re-opened, closed, rejected)
  - References (test case, requirement)
- Objectives of defect reports: give enough info to **resolve the issue**, provide a means of **tracking work-product quality**, and provide **ideas for improving** the development & test process.
- Static testing findings (review anomalies) are typically handled more simply/differently.

CHAPTER 6 — TEST TOOLS

6.1 Tool Support for Testing

Categories of tools (by activity supported):

- **Test management tools** — manage the SDLC, requirements, tests, defects, configuration (traceability!)
- **Static testing tools** — support reviews; static analysis tools
- **Test design & implementation tools** — generate test cases, test data, test procedures (e.g., model-based)
- **Test execution & coverage tools** — automated execution (functional test automation frameworks), coverage measurement
- **Non-functional testing tools** — performance/load testing, security scanning (test types hard to do manually)
- **DevOps tools** — CI/CD pipeline, build automation, delivery
- **Collaboration tools** — communication (wikis, chat, task boards)
- **Scalability & deployment standardization tools** — VMs, containerization (Docker, Kubernetes)
- Any other tool assisting testing (even a spreadsheet is a test tool in this sense!)

6.2 Benefits and Risks of Test Automation

Benefits:

- Saves time on **repetitive** manual work (regression runs, re-entering data, standards checking) → testers freed for higher-value work (new features, exploratory testing)
- **Greater consistency & repeatability** (no human variation; same data, same frequency/order)
- **Objective assessment** (coverage measures) and static measures (complexity)
- Easier access to testing information: statistics, graphs, aggregated data on progress, defects, durations, failure rates
- Reduced execution time → earlier defect detection, faster feedback, faster time-to-market
- Can do things impossible manually (e.g., simulate thousands of concurrent users in performance testing)

Risks:

- **Unrealistic expectations** about capability & ease of use
- Inaccurate estimation of time/cost/effort to **introduce** a tool, **maintain** the scripts, and change the manual process
- Using automation when manual testing is more appropriate
- Over-relying on tools (ignoring the need for human critical thinking; automated tests only check what they’re programmed to check)
- **Vendor dependence:** vendor may go out of business, retire the tool, sell it, offer poor support; open-source tool may be abandoned; tool incompatible with new platforms
- Legacy problems: tool or environment becomes unsupported; no upgrade path
- Test tools ≠ testing strategy — a tool won’t fix a poor process

QUICK-REVISION CHEAT SHEET (read this the morning of the exam)

- **Chain:** Root cause → Error (human) → Defect (in work product) → Failure (at execution)
- **7 principles:** presence not absence · exhaustive impossible · early testing · defect clustering · tests wear out (pesticide paradox) · context dependent · absence-of-defects fallacy
- **7 test activities:** Planning → Monitoring & Control → Analysis (WHAT) → Design (HOW) → Implementation (build testware) → Execution → Completion
- **Independence ladder:** author → peers → internal test team → external
- **Test levels (5):** Component → Component Integration → System → System Integration → Acceptance (UAT, operational, contractual/regulatory, alpha, beta)
- **Test types (4):** Functional (what) · Non-functional (how well) · Black-box · White-box — all applicable at every level
- **Confirmation testing** = verify the fix; **Regression testing** = verify nothing else broke
- **Maintenance triggers:** modification, environment migration/upgrade, retirement
- **Review process:** Planning → Initiation → Individual review → Communication & analysis → Fixing & reporting
- **Review types formality:** Informal < Walkthrough (author leads) < Technical review (moderator leads, technical experts) < Inspection (most formal, max anomalies, rules/checklists, metrics, author ≠ moderator/scribe/reader)
- **Review roles:** Manager, Author, Moderator, Scribe, Reviewer, Review leader
- **BVA:** 2-value = boundary + 1 neighbor; 3-value = boundary + both neighbors
- **Coverage subsumption:** 100% branch ⇒ 100% statement (not vice versa); all-transitions ⇒ valid-transitions ⇒ all-states
- **3 C's:** Card, Conversation, Confirmation · **INVEST:** Independent Negotiable Valuable Estimable Small Testable
- **Estimation:** ratios · extrapolation · Wideband Delphi / Planning Poker · three-point $E=(a+4m+b)/6$, $SD=(b-a)/6$
- **Test pyramid:** unit (many, fast) at bottom → UI/E2E (few, slow) at top
- **Quadrants:** Q1 tech/support (unit) · Q2 business/support (story tests) · Q3 business/critique (exploratory, usability) · Q4 tech/critique (performance, security)
- **Risk level = likelihood × impact** · Project risk = project objectives · Product risk = product quality
- **Severity** = impact on system · **Priority** = urgency to fix
- **Entry criteria ≈ Definition of Ready** · **Exit criteria ≈ Definition of Done**

FULL PRACTICE EXAM — 40 QUESTIONS (60 minutes, pass = 26 correct)

Chapter 1 (Q1–Q8)

Q1. Which of the following statements about testing is CORRECT? a) Testing is executing the software to show it works b) Testing includes activities such as planning, analysis, design and implementation, not only execution c) Testing and debugging are the same activity d) Testing can prove that the software has no defects

Q2. A developer misunderstood a requirement and wrote incorrect code. When a user runs that code, the application crashes. Which sequence is correct? a) The crash is a defect caused by an error in the code b) The misunderstanding is an error, the incorrect code is a defect, and the crash is a failure c) The misunderstanding is a failure that produced a defect d) The incorrect code is an error and the crash is a defect

Q3. Which testing principle explains why running the same regression suite repeatedly finds fewer and fewer new defects? a) Defect clustering b) Absence-of-defects fallacy c) Tests wear out d) Exhaustive testing is impossible

Q4. The application fulfilled 100% of its specified requirements and all reported defects were fixed, yet users rejected it because it does not support their actual workflow. Which principle does this illustrate? a) Testing is context dependent b) Absence-of-defects fallacy c) Early testing saves time and money d) Defects cluster together

Q5. During which test activity are test conditions identified and prioritized by analyzing the test basis? a) Test planning b) Test analysis c) Test design d) Test implementation

Q6. Which of the following is a benefit of test independence? a) Independent testers may challenge assumptions made by stakeholders during specification b) Independent testers are always more skilled than developers c) Developers retain full responsibility for quality d) Communication between testers and developers improves

Q7. Which of the following is a work product of test implementation? a) Prioritized test conditions b) Test completion report c) Test procedures and automated test scripts d) Defect reports for defects in the test basis

Q8. What distinguishes quality assurance (QA) from testing? a) QA focuses on fixing defects, testing focuses on preventing them b) QA is product-oriented, testing is process-oriented c) QA is process-oriented and preventive; testing is a form of quality control and is product-oriented d) QA is performed only by managers, testing only by testers

Chapter 2 (Q9–Q14)

Q9. Which practice is an example of shift-left? a) Running exploratory testing after release b) Writing test cases before the code is written and running them in a test harness during implementation c) Postponing non-functional testing until system testing d) Increasing the size of the independent test team

Q10. In BDD, desired behavior is typically expressed in which format? a) UML activity diagrams b) Given / When / Then scenarios understandable to stakeholders c) Pseudocode written by developers d) Decision tables signed off by management

Q11. Which test level focuses on interactions and interfaces between the whole system and other systems and external services? a) Component integration testing b) System testing c) System integration testing d) Acceptance testing

Q12. A tester runs a previously failed test to check that the defect fix works, and then runs a selection of other tests to check that the fix has not broken anything else. What are these two activities? a) Retesting, then confirmation testing b) Confirmation testing, then regression testing c) Regression testing, then confirmation testing d) Debugging, then retesting

Q13. A tax rate change in the law forces changes to a deployed payroll system. Testing of these changes is BEST described as: a) Corrective maintenance testing triggered by retirement b) Maintenance testing triggered by a modification (adapting to environment/regulation change) c) Beta testing d) Operational acceptance testing only

Q14. Which statement about test types and test levels is CORRECT? a) White-box testing can only be performed at component level b) Non-functional testing is only performed at system level c) Any test type can be performed at any test level d) Functional testing is only performed during acceptance testing

Chapter 3 (Q15–Q18)

Q15. Which defect is MOST likely to be found by static testing but NOT by dynamic testing? a) Slow response time under load b) A contradiction between two requirements in the specification c) A memory leak visible after 48 hours of operation d) Incorrect calculation result displayed to the user

Q16. Put the review process activities in the correct order:

1. Communication and analysis 2. Individual review 3. Fixing and reporting 4. Planning 5. Review initiation

a. 4, 5, 2, 1, 3
b. 4, 2, 5, 1, 3
c. 5, 4, 2, 3, 1
d. 4, 5, 1, 2, 3

Q17. In which review type does the AUTHOR typically lead the meeting, with a mandatory scribe and optional individual preparation? a) Inspection b) Technical review c) Walkthrough d) Informal review

Q18. Which of the following is a success factor for reviews? a) Using review results to evaluate the performance of the author b) Reviewing large documents in a single long session to save time c) Defining clear objectives and measurable exit criteria before the review d) Excluding testers to keep reviews technical

Chapter 4 (Q19–Q29)

Q19. A field accepts an integer number of passengers from 1 to 8. Using equivalence partitioning, what is the minimum number of test cases for 100% each-choice coverage? a) 2 b) 3 c) 4 d) 8

Q20. Same field (valid: integers 1–8). Which set of values achieves 100% **2-value** boundary value analysis coverage? a) 1, 8 b) 0, 1, 8, 9 c) 0, 1, 2, 7, 8, 9 d) 1, 4, 8

Q21. Same field. How many coverage-item values are required for 100% **3-value** BVA? a) 4 b) 5 c) 6 d) 8

Q22. A loyalty system: customers get a discount ONLY IF they hold a gold card AND the purchase exceeds $100. If they hold a silver card AND the purchase exceeds $100, they get free delivery. Which technique is MOST suitable to test the combinations of these conditions? a) Boundary value analysis b) Decision table testing c) State transition testing d) Statement testing

Q23. In a decision table, what does a dash "–" in a condition cell mean? a) The condition must be false b) The rule is infeasible c) The value of the condition is irrelevant to the outcome of this rule d) The action is not performed

Q24. A state model has 4 states and 7 valid transitions. A test suite exercises 6 of the valid transitions and visits all 4 states. What is the valid transitions coverage? a) 100% b) 85.7% c) 75% d) 57%

Q25. Which statement about coverage criteria in state transition testing is TRUE? a) All-states coverage guarantees valid-transitions coverage b) Valid-transitions coverage guarantees all-states coverage c) Valid-transitions coverage includes testing invalid transitions d) All-transitions coverage is weaker than all-states coverage

Q26. If a test suite achieves 100% branch coverage of the code, which statement is TRUE? a) It also achieves 100% statement coverage b) It guarantees all defects are found c) It also guarantees 100% of requirements are covered d) Statement coverage may still be below 100%

Q27. What is the fundamental LIMITATION of white-box testing? a) It cannot measure coverage b) It cannot detect defects of omission — requirements that were never implemented c) It can only be applied at system level d) It cannot be automated

Q28. A tester runs time-boxed sessions guided by test charters, simultaneously learning the product, designing and executing tests, and records findings in session sheets. This is: a) Checklist-based testing b) Error guessing c) Session-based exploratory testing d) Fault attack

Q29. Which set correctly lists the "3 C's" of user stories? a) Card, Conversation, Confirmation b) Code, Compile, Check c) Criteria, Conditions, Coverage d) Card, Criteria, Completion

Chapter 5 (Q30–Q38)

Q30. "All smoke tests must have passed and the test environment must be available before test execution starts." These are examples of: a) Exit criteria b) Entry criteria c) Test conditions d) Completion criteria

Q31. Using three-point estimation, the most optimistic estimate is 2 person-days, most likely is 5, most pessimistic is 14. What is the estimate E? a) 5 person-days b) 6 person-days c) 7 person-days d) 10 person-days

Q32. In the test pyramid, why are there more tests at the bottom layer than at the top? a) Bottom-layer tests are user-facing and therefore more valuable b) Bottom-layer (unit) tests are small, isolated, fast and cheap to run; top-layer tests are slow and complex c) Top-layer tests cannot be automated d) The pyramid mandates equal effort per layer

Q33. In the testing quadrants, exploratory testing and usability testing belong to: a) Q1 — technology-facing, supporting the team b) Q2 — business-facing, supporting the team c) Q3 — business-facing, critiquing the product d) Q4 —

technology-facing, critiquing the product

Q34. A key supplier may fail to deliver a required subsystem on time, delaying the whole project. This is: a) A product risk b) A project risk c) A defect d) A failure

Q35. Which activity belongs to product risk CONTROL rather than product risk analysis? a) Identifying potential failure modes of the product b) Assessing the likelihood and impact of each risk c) Monitoring whether mitigation actions are effective and identifying emerging risks d) Categorizing the identified risks

Q36. Which of the following would you expect in a test PROGRESS report but NOT typically emphasized in a test COMPLETION report? a) Lessons learned and reusable testware b) Testing planned for the next reporting period and current impediments c) Evaluation of product quality against objectives d) Unmitigated residual risks at release

Q37. What is the MAIN purpose of configuration management in testing? a) To automate the execution of regression tests b) To ensure all testware and test object items are uniquely identified, version controlled and traceable, so testing is reproducible c) To prioritize defects by severity d) To manage the testers' working hours

Q38. A defect report says: "Severity: high, Priority: low." Which situation fits this best? a) A typo on the login screen that must be fixed before tomorrow's demo b) A crash in a legacy report module that no customer has used for years c) A cosmetic misalignment that will never be fixed d) It is impossible — severity and priority must always match

Chapter 6 (Q39–Q40)

Q39. Which of the following is a BENEFIT of test automation? a) It removes the need for test planning b) It guarantees defect-free software c) Greater consistency and repeatability of test execution, and time saved on repetitive tasks d) It eliminates the need for human testers entirely

Q40. Which of the following is a RISK of using test execution tools? a) Tests are executed with greater consistency b) Objective measures such as coverage become available c) The vendor may go out of business or offer poor support, and maintenance effort for scripts may be underestimated d) Feedback on quality is provided faster

---

ANSWER KEY WITH EXPLANATIONS

Q1 — b. Testing is a set of activities including planning, monitoring, analysis, design, implementation, execution and completion; execution is only one part. (a) is too narrow, (c) confuses testing with debugging, (d) violates principle 1.

Q2 — b. Human mistake = error → produces defect in code → executing the defect = failure.

Q3 — c. "Tests wear out" (pesticide paradox): repeated identical tests stop finding new defects; tests must be revised.

Q4 — b. Absence-of-defects fallacy: fulfilling all requirements and fixing all defects doesn't guarantee the product meets users' real needs.

Q5 — b. Test analysis answers "what to test" and produces prioritized test conditions. Test design answers "how to test" (test cases).

Q6 — a. Recognized benefit of independence. (d) is the opposite — communication problems are a drawback; (c) is a drawback (developers may LOSE the sense of responsibility); (b) is false.

Q7 — c. Implementation produces test procedures, scripts, suites, test data, environment elements. Test conditions come from analysis; completion report from completion.

Q8 — c. QA = process-oriented, preventive; testing = quality control, product-oriented, corrective.

Q9 — b. Test-first with a harness during implementation is a classic shift-left example.

Q10 — b. BDD uses Given/When/Then (Gherkin) scenarios understandable to all stakeholders.

Q11 — c. System integration testing covers interfaces between the system and other systems/external services. Component integration is between components inside the system.

Q12 — b. First re-running the failed test = confirmation testing (retesting); checking for side effects = regression testing.

Q13 — b. A legal/regulatory change forcing modification of an operational system triggers maintenance testing (adaptive maintenance).

Q14 — c. All four test types (functional, non-functional, black-box, white-box) can be applied at every test level.

Q15 — b. Defects in non-executable work products (requirement contradictions, ambiguities, omissions) can only be found by static testing. a, c, d require execution.

Q16 — a. Planning → Review initiation → Individual review → Communication & analysis → Fixing & reporting.

Q17 — c. Walkthrough: led by the author, scribe mandatory, individual preparation optional.

Q18 — c. Clear objectives and measurable exit criteria are a success factor. Evaluating participants (a) must never be an objective; (b) overloads reviewers; (d) excludes valuable reviewers.

Q19 — b. Partitions: <1 invalid, 1–8 valid, >8 invalid → 3 partitions → minimum 3 tests (one value per partition), e.g., 0, 4, 9.

Q20 — b. 2-value BVA of range 1–8: boundaries 1 and 8, plus nearest neighbors in adjacent partitions: 0 and 9 → {0, 1, 8, 9}.

Q21 — c. 3-value BVA: each boundary plus both neighbors: 0,1,2 and 7,8,9 → 6 values.

Q22 — b. Behavior depends on combinations of conditions (card type × amount) → decision table testing.

Q23 — c. A dash means "don't care" — the condition's value doesn't affect the outcome for that rule (used when collapsing the table).

Q24 — b. Valid transitions coverage = exercised valid transitions ÷ total valid transitions = 6/7 ≈ 85.7%. Visiting all states is irrelevant to this metric.

Q25 — b. Achieving full valid-transitions (0-switch) coverage guarantees full all-states coverage. All-transitions (valid + invalid) is the strongest of the three.

Q26 — a. Branch coverage subsumes statement coverage: 100% branch ⇒ 100% statement. It never guarantees all defects (b) or requirements coverage (c).

Q27 — b. White-box tests only what exists in the code; unimplemented requirements (defects of omission) go

undetected.

Q28 — c. Charters + time-boxed sessions + concurrent learn/design/execute + session sheets = session-based exploratory testing.

Q29 — a. Card, Conversation, Confirmation.

Q30 — b. Preconditions to START an activity = entry criteria (Definition of Ready in Agile).

Q31 — b. E = (a + 4m + b)/6 = (2 + 20 + 14)/6 = 36/6 = 6 person-days. (SD = (14−2)/6 = 2.)

Q32 — b. Unit tests are small, isolated, fast, cheap → many of them; E2E/UI tests are slow, complex, brittle → few.

Q33 — c. Q3 = business-facing, critique the product: exploratory, usability, UAT.

Q34 — b. Supplier delivery failure threatening the schedule affects project objectives → project risk. Product risks concern the quality of the product itself.

Q35 — c. Risk control = mitigation + monitoring. Identification, assessment (likelihood/impact), categorization belong to risk analysis.

Q36 — b. Progress reports (during testing) include impediments and next-period plans. Completion reports (after) include lessons learned, quality evaluation, residual risks.

Q37 — b. CM ensures unique identification, version control, change tracking and traceability of the test object and all testware → reproducible testing.

Q38 — b. Severity (technical impact: a crash = high) can differ from priority (business urgency: unused module = low). They are independent attributes.

Q39 — c. Consistency, repeatability, and time saved on repetitive work are core automation benefits. Automation never guarantees defect-free software or replaces planning/humans.

Q40 — c. Vendor dependence and underestimated introduction/maintenance effort are classic tool risks. a, b, d are benefits.

SCORING

26+/40 → you would PASS the real exam.

- 30–34: solid — review the chapters of any questions you missed.
- 26–29: passing but shaky — re-read Chapters 4 and 5 (they carry half the exam).
- <26: focus study on the 7 principles, test levels/types, BVA/EP/decision tables/state transitions (K3 — practice applying them!), estimation formula, and severity vs. priority.

FINAL EXAM-DAY TIPS

1. Chapter 4 + Chapter 5 = 20 of the 40 questions. Master the K3 techniques (EP, BVA, decision tables, state transitions, ATDD) — you WILL get scenario questions requiring calculation.
2. Memorize the three-point formula: E = (a+4m+b)/6.
3. Watch for “NOT” and “EXCEPT” in question stems — read every question twice.
4. Two answers often look right — pick the one using exact syllabus terminology.
5. Never leave a blank; there is no negative marking.
6. Budget ~90 seconds per question; flag hard ones and return.